R3.05 · TD-TP 05

PROGRAMMATION SYSTÈME

Les fichiers

BUT2 Informatique · R3.05 · 2026-2027 · Séance de 2 h

Opération « Coffre-fort ». Un fichier banal — une carte postale de Dijon — cache un second fichier. Pour l'ouvrir, il te faudra maîtriser les flots de la bibliothèque standard, le positionnement dans un fichier et les appels système qui travaillent sous le capot.

OBJECTIFS DE LA SÉANCE

- ☐ Ouvrir, lire, écrire et fermer un fichier avec la bibliothèque standard (`fopen`, `fgetc`, `fread`, `fprintf`, `fclose`).
- ☐ Distinguer fichier texte et fichier binaire ; lire et écrire des données brutes avec `fread`/`fwrite`.
- ☐ Se positionner dans un fichier : `fseek`, `ftell`, `rewind`.
- ☐ Refaire les mêmes opérations avec les **appels système** du CM 02 : `open`, `read`, `write`, `lseek`, `close`, et mesurer la différence avec `strace`.
- ☐ Créer un fichier **en sécurité** : tests d'erreur systématiques, droits et `umask`, `O_EXCL`, piège TOCTOU.

FIL ROUGE Opération « Coffre-fort »

Le LAMAE a intercepté trois fichiers : `briefing.txt`, `miroir.txt` et `carte_postale.txt` (dossier `code/sujet05/`). Chaque mission fait apparaître un **drapeau** `FLAG{...}` : notez-les dans `badge05.txt`. Le dernier se trouve **dans** la carte postale, invisible avec un simple `cat`. Cotation : ★ incontournable, ★★ cœur de séance, ★★★ pour aller loin.

Mission	Contenu	Durée
1 · Le flot	`FILE *`, `fopen`/`fclose`, `fgetc`/`fputc` : la commande `cat`	20 min
2 · Le tapis roulant	`fread`/`fwrite`, tampons : la commande `cp`	20 min
3 · Le curseur	`fseek`, `ftell`, `rewind` : lecture inversée	15 min
4 · Sous le capot	`open`, `read`, `write`, `close`, `strace`, droits et `umask`	25 min
5 · Le coffre	Stéganographie : cacher et extraire un fichier	25 min
6 · Le coffre suspect	Sécurité : `O_EXCL`, TOCTOU, `/tmp`	15 min
Bonus	Pour les rapides : mini `ls -li`, mini `xxd`	—

compilation en R3.05

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g prog.c -o prog
```

> **PIÈGE CLASSIQUE** **-std=c17 et les fonctions POSIX**
>
> En `-std=c17`, gcc ne déclare que le C **standard**. Les fonctions POSIX (`open`, `read`, `write`, `lseek`, `fileno`, `stat`, `mkstemp`...) peuvent alors être absentes des en-têtes (« implicit declaration of function »). Convention du module : ajoutez **en toute première ligne**, avant tout `#include`, la ligne `#define _POSIX_C_SOURCE 200809L` dans chaque programme qui utilise des fonctions POSIX. Les programmes qui n'utilisent que `stdio.h` n'en ont pas besoin.

Mission 1 — Le flot : ouvrir, lire, fermer

Le C permet de lire et d'écrire des données dans un fichier. Pour des raisons d'efficacité, les accès passent par une **mémoire tampon** (*buffer*), ce qui réduit le nombre d'accès au périphérique (disque...). Pour manipuler un fichier, un programme a besoin de plusieurs informations :

1. l'adresse de la mémoire tampon ;
2. le fichier concerné (en réalité, son **descripteur**, voir mission 4) ;
3. la position courante de la tête de lecture/écriture ;
4. le mode d'accès (lecture ou écriture) ;
5. les indicateurs d'erreur et de fin de fichier.

Ces informations sont rassemblées dans une structure de type `FILE`, définie dans `stdio.h`. Un pointeur `FILE *` est appelé **flot de données** (*stream*). On l'obtient avec `fopen`, qui négocie avec le système d'exploitation ; après les traitements, `fclose` rompt la liaison entre le fichier et le flot.

Les fonctions fopen et fclose

```c
FILE *fopen(const char *nomFichier, const char *mode);
int fclose(FILE *flot);    /* 0 si tout va bien, EOF en cas d'erreur */
```

`fopen` renvoie le flot, ou `NULL` en cas d'échec (fichier inexistant, droits insuffisants...) : il faut **toujours** tester ce retour. On préférera définir le nom du fichier par une constante (`#define`) ou le recevoir en argument (`argv`), plutôt que de l'écrire en dur au milieu du code.

Mode	Ouverture
`"r"`	lecture ; le fichier **doit exister**
`"w"`	écriture ; créé s'il n'existe pas, **vidé** s'il existe (ancien contenu perdu)
`"a"`	écriture à la fin (*append*) ; créé s'il n'existe pas
`"r+"` `"w+"` `"a+"`	lecture et écriture, avec les mêmes règles de création que `r`, `w`, `a`
`b` ajouté	`"rb"`, `"wb"`, `"ab"`, `"r+b"`... : mode **binaire**, à utiliser dès qu'on manipule des données non textuelles

Sous Linux, texte et binaire sont identiques ; sous Windows, le mode texte convertit les fins de ligne `\r\n`. Mettre `b` pour un fichier binaire rend donc le programme **portable**.

Trois flots standard sont ouverts automatiquement : `stdin` (entrée, par défaut le clavier), `stdout` (sortie, l'écran) et `stderr` (erreurs, l'écran, **non tamponnée**). Affichez toujours les messages d'erreur sur `stderr`, idéalement avec `perror`, qui ajoute la raison de l'échec (`errno`) : ils restent visibles même si la sortie standard est redirigée vers un fichier.

Caractère par caractère : fgetc et fputc

```c
int fgetc(FILE *flot);           /* caractère lu, ou EOF (fin de fichier ou erreur) */
int fputc(int caractere, FILE *flot); /* caractère écrit, ou EOF en cas d'erreur */
```

`getc` et `putc` font la même chose (elles peuvent être implémentées par des macros). Comme pour `getchar`, la variable qui reçoit le résultat de `fgetc` doit être un `int` pour pouvoir distinguer `EOF` des 256 valeurs d'octet. Le programme suivant (fichier `code/sujet05/copie.c`) recopie `entree.txt` dans `sortie.txt` :

copie.c

```c
#include <stdio.h>
#include <stdlib.h>

#define ENTREE "entree.txt"
#define SORTIE "sortie.txt"

int main(void)
{
    FILE *fIn, *fOut;
    int c;                                  // int, pas char : EOF !

    if ((fIn = fopen(ENTREE, "r")) == NULL) {
        perror(ENTREE);                     // message + raison, sur stderr
        return EXIT_FAILURE;
    }
    if ((fOut = fopen(SORTIE, "w")) == NULL) {
        perror(SORTIE);
        fclose(fIn);
        return EXIT_FAILURE;
    }
    while ((c = fgetc(fIn)) != EOF)
        fputc(c, fOut);

    fclose(fIn);
    if (fclose(fOut) == EOF) {              // l'écriture réelle peut échouer ici
        perror("fclose");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

Entrées-sorties formatées : fprintf et fscanf

`fprintf(flot, "format", ...)` et `fscanf(flot, "format", ...)` fonctionnent exactement comme `printf` et `scanf`, mais sur le flot de votre choix. `printf(...)` n'est qu'un raccourci pour `fprintf(stdout, ...)`. Comme `scanf`, `fscanf` renvoie le nombre de conversions réussies : testez-le.

EX 1 La commande cat

★ 15 min

Écrivez `mon_cat`, qui affiche sur la sortie standard le contenu de chacun des fichiers passés en arguments, avec `fopen`, `fgetc` et `fputc`.

TERMINAL

```
$ ./mon_cat briefing.txt
$ ./mon_cat briefing.txt inexistant.txt miroir.txt
$ ./mon_cat < briefing.txt        # sans argument : lire stdin, comme cat
```

1. Si un fichier n'existe pas, affichez un message avec `perror` et **passez au suivant** ; le code de retour final doit valoir `EXIT_FAILURE` (vérifiez avec `echo $?`).

2. `fgetc` renvoie `EOF` à la fin du fichier **ou** en cas d'erreur de lecture : comment les distinguer ? (`man 3 ferror`)
3. **Drapeau :** le premier fragment se trouve à la fin de `briefing.txt`.

Mission 2 — Le tapis roulant : fread, fwrite et tampons

Les entrées-sorties **binaires** transfèrent des octets sans aucune conversion. Elles sont plus efficaces que les entrées-sorties formatées, mais les fichiers produits ne sont pas portables : le codage des données (taille d'un `int`, ordre des octets) dépend de la machine. Elles sont utiles pour les données volumineuses ou de type composé (tableaux, structures).

```c
size_t fread(void *ptr, size_t taille, size_t nombre, FILE *flot);
size_t fwrite(const void *ptr, size_t taille, size_t nombre, FILE *flot);
```

`ptr` est l'adresse des données, `taille` la taille d'un élément, `nombre` le nombre d'éléments. Les deux fonctions renvoient le **nombre d'éléments** effectivement transférés, qui peut être inférieur à `nombre` (fin de fichier, erreur) : c'est cette valeur qu'il faut utiliser. Le type `size_t`, défini dans `stddef.h`, est le type entier non signé du résultat de `sizeof`.

L'exemple `code/sujet05/positionnement.c` écrit les entiers de 0 à 49 dans le fichier binaire `tresor.bin` avec `fwrite`, puis les relit avec `fread`. Après exécution, `cat tresor.bin` n'affiche que des caractères bizarres : le fichier contient les octets **bruts** des `int`, pas leur écriture décimale. Pour les voir : `od -A d -t d4 tresor.bin` ou `xxd tresor.bin`.

EX 2 La commande cp

★★ 20 min

Écrivez `mon_cp source destination [taille_tampon]` avec la bibliothèque standard.

1. Version 1 : copie caractère par caractère (`fgetc`/`fputc`).
2. Version 2 : copie par blocs avec `fread`/`fwrite` et un tampon de 4 octets, puis de 1024 octets. **Écrivez exactement le nombre d'octets lus** (valeur de retour de `fread`), pas la taille du tampon : sinon la fin du fichier copié contient des déchets.
3. La taille du tampon est passée en 3e argument : convertissez-la avec `strtol` (pas `atoi`, qui ne détecte aucune erreur) et refusez une valeur négative, nulle, non numérique ou absurdement grande.
4. Créez un gros fichier de test et comparez les temps :

```
TERMINAL
$ head -c 1000000 /dev/urandom > gros.bin      # 1 000 000 octets aléatoires
$ time ./mon_cp gros.bin copie1.bin 4
$ time ./mon_cp gros.bin copie2.bin 1024
$ cmp gros.bin copie2.bin && echo identiques
```

Drapeau : `FLAG{` + le nombre d'appels à `fread` qui renvoient une valeur non nulle pour copier `gros.bin` avec un tampon de 1024 octets + `}`.

> **INDICE**
>
> `fopen` en `"rb"`/`"wb"` : un fichier quelconque (image, exécutable) n'est pas du texte. Et n'oubliez pas de tester le retour de `fclose` sur le fichier écrit : c'est souvent là que l'écriture sur le disque échoue (disque plein, quota).

Mission 3 — Le curseur : se positionner dans un fichier

Les fonctions précédentes accèdent au fichier en mode **séquentiel** : les données sont lues ou écrites les unes à la suite des autres. On peut aussi y accéder en mode **direct** (aléatoire), en se positionnant à n'importe quel endroit.

```c
int fseek(FILE *flot, long deplacement, int origine);  /* 0 si succès, -1 sinon */
long ftell(FILE *flot);                                /* position courante en octets */
void rewind(FILE *flot);                               /* retour au début */
```

`deplacement` est compté en octets, relativement à l'`origine` : `SEEK_SET` (début du fichier), `SEEK_CUR` (position courante) ou `SEEK_END` (fin du fichier). `rewind(flot)` équivaut à `fseek(flot, 0, SEEK_SET)` (et efface en plus l'indicateur d'erreur). Extrait de `positionnement.c` :

```c
fseek(fIn, 0, SEEK_END);                          // fin du fichier
printf("position %ld\n", ftell(fIn));
fseek(fIn, -10L * (long) sizeof(int), SEEK_END);  // 10 int en arrière
printf("position %ld", ftell(fIn));
fread(&i, sizeof i, 1, fIn);
printf("\t i = %d", i);
rewind(fIn);                                      // retour au début
printf("\nposition %ld", ftell(fIn));
fread(&i, sizeof i, 1, fIn);
printf("\t i = %d", i);
fseek(fIn, 5L * (long) sizeof(int), SEEK_CUR);    // 5 int en avant
printf("\nposition %ld", ftell(fIn));
fread(&i, sizeof i, 1, fIn);
printf("\t i = %d\n", i);
```

```
$ ./positionnement
position 200
position 160     i = 40
position 0       i = 0
position 24      i = 6
```

On constate que `fread` déplace la position courante de la taille de l'objet lu.

> **PIÈGE CLASSIQUE** **Le déplacement négatif et sizeof**
>
> L'énoncé historique écrivait `fseek(f, -10 * sizeof(int), SEEK_END)`. Or `sizeof` est de type `size_t`, **non signé** : `-10 * sizeof(int)` vaut 18 446 744 073 709 551 576, converti ensuite en `long`. Cela « tombe juste » sur Linux 64 bits, mais c'est un hasard de représentation. Écrivez `-10L * (long) sizeof(int)`.

EX 3 — Lecture inversée

★ 15 min

Écrivez `inverse fichier`, qui affiche le contenu du fichier à l'envers : si le fichier contient `abcdef`, le programme affiche `fedcba`. Placez-vous à la fin du fichier, puis lisez caractère par caractère, du dernier vers le premier.

1. Testez chaque `fseek`. Que se passe-t-il pour un fichier vide ?
2. **Drapeau :** appliquez votre programme à `miroir.txt`.
3. **Drapeau :** quelle position affiche `positionnement` après le déplacement de 10 `int` en arrière ? Écrivez `FLAG{` + cette valeur + `}`.

> **POUR LES RAPIDES** **Et les caractères accentués ?**
> Inversez un fichier contenant `été`. Pourquoi le résultat est-il illisible ? (En UTF-8, `é` occupe 2 octets ; votre programme inverse des **octets**, pas des caractères.)

Mission 4 — Sous le capot : les appels système

`fopen`, `fgetc` ou `fread` sont des fonctions de la **bibliothèque** C : elles ne parlent pas directement au disque. Pour cela, elles appellent le noyau par des **appels système** (CM 02), qui travaillent sur des **descripteurs** : de simples entiers, indices dans la table des descripteurs du processus (0 = `stdin`, 1 = `stdout`, 2 = `stderr`).

Appel système	Rôle
`int open(const char *chemin, int drapeaux, mode_t mode);`	Ouvre (ou crée) un fichier, renvoie le **plus petit** descripteur libre ou -1.
`ssize_t read(int fd, void *buf, size_t nb);`	Lit **au plus** `nb` octets ; renvoie le nombre lu, 0 en fin de fichier, -1 en cas d'erreur.
`ssize_t write(int fd, const void *buf, size_t nb);`	Écrit **au plus** `nb` octets ; renvoie le nombre écrit ou -1.
`off_t lseek(int fd, off_t depl, int origine);`	Déplace la position courante (mêmes origines que `fseek`).
`int close(int fd);`	Libère le descripteur ; 0 ou -1.

Les drapeaux de `open` se combinent avec `|` : un seul parmi `O_RDONLY`, `O_WRONLY`, `O_RDWR`, avec éventuellement `O_CREAT` (créer si absent), `O_TRUNC` (vider), `O_APPEND` (écrire toujours à la fin), `O_EXCL` (avec `O_CREAT` : échouer si le fichier existe). En-têtes : `<fcntl.h>` pour `open`, `<unistd.h>` pour `read`, `write`, `lseek`, `close`.

> **SÉCURITÉ** **open avec O_CREAT : le mode est obligatoire**
> `open("notes.txt", O_WRONLY | O_CREAT | O_TRUNC)` sans 3e argument est un bug classique : `open` lit alors une valeur **quelconque** sur la pile comme droits du fichier créé (parfois `rwsrwxrwx`...). Avec `O_CREAT`, donnez **toujours** le mode : `0644` pour un fichier ordinaire, `0600` pour un fichier confidentiel. Les droits réellement obtenus valent `mode & ~umask`.

EX 4 — cp, version noyau

★★ 25 min

Complétez `code/sujet05/cp_sys.c` : `cp_sys source destination [taille_tampon]`, avec `open`, `read`, `write` et `close` uniquement (pas de `FILE *`).

1. Chaque appel système a son test d'erreur (`perror`). Le cas `read` renvoyant 0 est la **fin du fichier**, pas une erreur.
2. `write` peut écrire **moins** d'octets que demandé (disque presque plein, tube, socket, signal) : écrivez la fonction `ecrireTout` qui boucle jusqu'à ce que tout soit écrit.
3. Comptez les appels système avec `strace` (installation : `sudo apt install strace`) pour des tampons de 1, 1024 et 65536 octets, et comparez avec `mon_cp` de la mission 2 en tampon de 4 octets (commandes ci-dessous).
4. Pourquoi `mon_cp` avec un tampon de 4 octets fait-il pourtant si peu d'appels `read` ? (Indice : le tampon caché du `FILE`, de 4096 octets en général.)
5. Droits et `umask` : tapez `umask 027`, copiez `briefing.txt` avec `cp_sys` (création en `0644`), puis recommencez après avoir mis le mode `0666` dans le code, et regardez les droits obtenus avec `ls -l` et `stat -c '%a %i' fichier` (droits en octal et numéro d'**i-nœud**). Revenez ensuite à `umask 022`.

```
TERMINAL
$ strace -c -e trace=read,write ./cp_sys gros.bin c3.bin 1
$ strace -c -e trace=read,write ./cp_sys gros.bin c3.bin 1024
$ strace -c -e trace=read,write ./mon_cp gros.bin c4.bin 4
$ umask 027 ; ./cp_sys briefing.txt b.txt ; stat -c '%a %i' b.txt ; umask 022
```

Drapeau : `FLAG{` + les droits octaux (4 chiffres) d'un fichier créé avec le mode `0666` sous `umask 027` + `}`.

> **CULTURE** **Le tampon de stdio et les appels système**
>
> Avec un tampon de 1 octet, `cp_sys` fait un million d'appels `read` et autant de `write` : chacun est un passage en mode noyau, coûteux. `fgetc`, lui, lit 4096 octets d'un coup dans son tampon puis vous les distribue un par un : c'est tout l'intérêt de `stdio`. `fflush(flot)` force la vidange du tampon ; `setvbuf` en change la taille.

Mission 5 — Le coffre : stéganographie

La **stéganographie** consiste à cacher un message dans un support anodin. Version simple : on ajoute un fichier secret **à la fin** d'un fichier porteur. Le porteur reste lisible normalement (un lecteur de texte ou d'image s'arrête souvent avant les données ajoutées) et le secret passe inaperçu. Pour le récupérer, on enregistre la taille du secret à la toute fin du fichier ; à l'extraction, on lit d'abord cette taille, puis on se positionne au bon endroit.

Format imposé au LAMAE (8 octets de signature, puis la taille sur 8 octets, octet de poids faible en premier) :

```
structure du porteur après cacher
[ contenu d'origine ][ contenu caché : n octets ][ "LAMAE05\0" : 8 octets ][ n : 8 octets ]
```

EX 5 Cacher et extraire

★★ 25 min

Complétez `code/sujet05/steg.c` (les fonctions `ecrireU64` et `lireU64` sont fournies).

1. `./steg cacher secret porteur` : ouvrez `porteur` en `"ab"`, recopiez-y `secret` en comptant les octets, puis écrivez la signature et la taille.
2. `./steg extraire porteur sortie` : `fseek` à 16 octets de la fin, lecture et **vérification** de la signature, lecture de la taille `n`, puis `fseek` au début du contenu caché et recopie de `n` octets.
3. Testez sur vos propres fichiers, puis vérifiez la structure avec `xxd porteur | tail -3`.
4. **Drapeau :** extrayez le fichier caché dans `carte_postale.txt`.

> **SÉCURITÉ** **Ne jamais faire confiance à une taille lue dans un fichier**
>
> Le fichier porteur vient de l'extérieur : un adversaire peut y écrire la taille qu'il veut. Avant de l'utiliser pour un `fseek`, un `malloc` ou une boucle, vérifiez que `n <= taille_du_fichier - 16`. Testez votre programme avec ce fichier piégé, il doit refuser proprement :
>
> ```
> TERMINAL
> $ cp briefing.txt piege.bin
> $ printf 'LAMAE05\0\xff\xff\xff\xff\xff\xff\xff\x7f' >> piege.bin
> $ ./steg extraire piege.bin sortie.txt
> ```
>
> Ce type de faille (une longueur venant des données, non validée) est à l'origine de nombreuses vulnérabilités réelles dans des lecteurs d'images, d'archives ou de protocoles réseau (Heartbleed en 2014).

PIÈGE CLASSIQUE **Pourquoi pas fwrite(&n, sizeof n, 1, f) ?**

Ce serait plus court, mais le fichier dépendrait de la machine : taille du type et **boutisme** (ordre des octets). Un coffre fabriqué sur un processeur grand-boutiste ne s'ouvrirait pas sur un PC. Écrire les octets un par un, dans un ordre fixé, rend le format portable.

Mission 6 — Le coffre suspect : sécurité des fichiers

Un stagiaire a écrit `code/sujet05/coffre_suspect.c`, qui range un secret dans `/tmp/coffre_lamae.txt`. Il compile sans avertissement… et contient pourtant trois défauts de sécurité.

coffre_suspect.c (extrait)

```c
#define CHEMIN "/tmp/coffre_lamae.txt"
if (access(CHEMIN, F_OK) == 0) {          // le fichier existe-t-il déjà ?
    fprintf(stderr, "Le coffre existe déjà\n");
    return EXIT_FAILURE;
}
/* ... fenêtre de temps entre la vérification et l'utilisation ... */
int fd = open(CHEMIN, O_WRONLY | O_CREAT | O_TRUNC, 0666);
```

EX 6 Trois failles, trois correctifs ★★ 15 min

1. **TOCTOU** (*Time Of Check to Time Of Use*) : entre `access` et `open`, un autre utilisateur peut créer `/tmp/coffre_lamae.txt` sous forme de **lien symbolique** vers un de vos fichiers (`ln -s ~/.bashrc /tmp/coffre_lamae.txt`). Que fait alors `open` avec `O_TRUNC` ? Remplacez le couple `access` + `open` par un **seul** `open` avec `O_CREAT | O_EXCL` (création atomique : échec avec `EEXIST` si le nom existe, y compris sous forme de lien) et ajoutez `O_NOFOLLOW`.
2. **Droits** : avec `0666` et l'`umask` habituel 022, qui peut lire le secret ? Choisissez le bon mode.
3. **Nom prévisible** dans `/tmp` : `/tmp` est partagé par tous les utilisateurs. Pour un fichier temporaire, utilisez `mkstemp` (`man 3 mkstemp`), qui crée un nom unique et imprévisible, en `0600`.
4. Le retour de `write` et celui de `close` ne sont pas vérifiés : corrigez.

Drapeau : `FLAG{` + le nom du drapeau de `open` qui rend la création atomique + `}`.

GUIDE ANSSI **Règles du guide ANSSI**

Le guide *Règles de programmation pour le développement sécurisé de logiciels en langage C* demande notamment de **vérifier systématiquement les valeurs de retour** des fonctions et appels système, et de ne pas utiliser de données externes sans validation. Le principe du **moindre privilège** s'applique aussi aux fichiers : les droits les plus restrictifs possibles.

POUR LES RAPIDES **Mini ls -li et mini xxd ★★★**

1. Écrivez `mini_ls fichier...` qui affiche, pour chaque argument, le numéro d'i-nœud, le type (`-`, `d`, `l`, `c`, `b`, `p`, `s`), les droits `rwxr-xr-x`, le nombre de liens, l'UID et la taille, avec `lstat` (`man 2 stat`, macros `S_ISREG`, `S_ISDIR`…). Pourquoi `lstat` plutôt que `stat` pour un lien symbolique ?
2. Écrivez `mini_xxd fichier` : 16 octets par ligne, la position en hexadécimal, les octets en hexadécimal (`%02x`), puis les caractères imprimables (`isprint`), un point sinon. Pourquoi ne faut-il jamais envoyer des octets de contrôle bruts au terminal ?
3. Réécrivez `inverse` avec `open`, `lseek` et `read` : combien d'appels système pour un fichier de 1 Mo ? Proposez une version qui lit le fichier **par blocs** depuis la fin.

Bilan de la séance

À RETENIR

- `FILE *` (bibliothèque, tamponné) et descripteur `int` (noyau) : `fopen`/`fread` s'appuient sur `open`/`read`.
- **Chaque appel, son test d'erreur** : `fopen` → `NULL`, appels système → -1 et `errno`, `fread`/`read` → nombre réellement lu, `fclose`/`close` → aussi.
- `read`/`write` peuvent transférer **moins** que demandé : on boucle.
- `O_CREAT` exige un mode ; droits obtenus = `mode & ~umask`.
- Créer un fichier sensible : `O_CREAT | O_EXCL`, droits `0600`, `mkstemp` dans `/tmp`, jamais `access` puis `open`.
- Une taille lue dans un fichier est une donnée **hostile** tant qu'elle n'a pas été validée.

- ☐ J'ai mes 7 drapeaux dans `badge05.txt`.
- ☐ Tous mes programmes compilent sans avertissement, avec `#define _POSIX_C_SOURCE 200809L` quand ils utilisent des fonctions POSIX.

Aller plus loin

Ressources

Ressource	Pour quoi faire
`man 3 fopen`, `man 3 fread`, `man 3 fseek`, `man 3 setvbuf`	La bibliothèque standard d'entrées-sorties.
`man 2 open`, `man 2 read`, `man 2 write`, `man 2 lseek`, `man 2 close`	Les appels système (section 2) : drapeaux, erreurs possibles (`ERRORS`).
`man 2 stat`, `man 7 inode`, `man 2 umask`, `man 3 mkstemp`	Attributs, i-nœuds, droits, fichiers temporaires sûrs.
`man 3 errno`, `man 3 perror`	Codes d'erreur (`ENOENT`, `EACCES`, `EEXIST`...).
cppreference — E/S de fichiers en C	Référence normalisée de `stdio.h`.
M. Kerrisk, *The Linux Programming Interface* (No Starch Press), chap. 4-5 (E/S), 13 (tampons), 14-15 (systèmes de fichiers, attributs)	La référence pour tout ce qui touche aux fichiers sous Linux.
A. Tanenbaum, *Systèmes d'exploitation*, chapitre sur les systèmes de fichiers	Allocation des blocs, i-nœuds, journalisation.
J.-M. Rifflet, *La programmation sous UNIX*	Les primitives d'E/S expliquées en français.
Guide ANSSI (dossier du CM)	Vérification des retours, validation des entrées, moindre privilège.
`strace`, `ltrace`	`strace` montre les appels système d'un programme ; `ltrace` les appels de bibliothèque (`fopen`, `fread`...).
`xxd`, `od`, `hexdump`, `stat`, `ls -li`	Observer le contenu brut et les attributs d'un fichier.

Pour approfondir

CULTURE **Tout est fichier**

Sous UNIX, les mêmes appels `open`, `read`, `write` et `close` servent pour les fichiers ordinaires, les périphériques (`/dev/tty`, `/dev/urandom`), les tubes (sujet 07) et même les connexions réseau (sujet 09). Essayez `./cp_sys /dev/urandom alea.bin 16`... puis interrompez avec Ctrl+C, car ce fichier n'a pas de fin !

CULTURE **Écrire n'est pas enregistrer**

Un `write` réussi signifie que les données sont dans le **cache du noyau**, pas sur le disque. En cas de coupure de courant, elles peuvent être perdues. Les bases de données appellent `fsync(fd)` pour forcer l'écriture physique, et écrivent dans un fichier temporaire qu'elles renomment ensuite avec `rename` (opération atomique) pour ne jamais laisser un fichier à moitié écrit.

CULTURE **TOCTOU, un classique**

Les attaques par lien symbolique dans `/tmp` ont touché de nombreux programmes privilégiés (outils d'impression, compilateurs, scripts d'installation). Linux a ajouté la protection `fs.protected_symlinks` (`sysctl fs.protected_symlinks`) pour limiter les dégâts, mais la bonne réponse reste dans le code : `O_EXCL`, `O_NOFOLLOW`, `mkstemp`, et les appels relatifs à un descripteur de répertoire (`openat`).